

Corsaire-1 Technical Report

BlaiseAI Research

Draft date: May 11, 2026

Compressed SFT for a REAP-pruned DeepSeek-V3.2 MoE under NVFP4 W4A4/KV4 training constraints

This draft emphasizes BlaiseAI's training-system contributions: low-bit optimizer state, transient-master updates, activation residual correction, trainable cache quantizers, long-context replay, and B200 kernel integration.

Contents

Abstract	4
1. Introduction	4
Contributions	4
2. Base Model, Data, and Training Target	5
2.1 Model Lineage	5
2.2 Architecture Snapshot	5
2.3 Data	6
3. Numerical Design	6
3.1 NVFP4 W4A4 as the Shared Compute Substrate	7
3.2 SpinQuant Rotations	7
3.3 TurboQuant for the Dense MLA Latent	7
3.4 IndexCache-Style FP8 DSA Indexer Key Quantization	8
4. Architectural Additions	8
4.1 GatedNorm	8
4.2 G1 Attention Output Gate	9
5. Optimizer and Error Feedback	9
5.1 FlashAdamW State Compression	9
5.2 Weight-Side ECO and the Transient Master	10
5.3 Activation Residual Compensation (ARC)	10
5.4 Gradient Release and Bucket-Completion Overlap	11
5.5 Autotune Correctness	11
6. Long-Context Systems Work	11
6.1 StreamBP	11
6.2 DSA Kernels and Sparse-Attention Bottlenecks	12
6.3 B200 CuTe Kernel Ports	12
6.4 Zero-Cost Checkpointing	12
7. Training Configuration	13
8. Validation	14
8.1 Correctness Surface	14
8.2 Microbenchmarks	14

8.3 Short-Run Training Probes 15

9. Discussion 15

9.1 Why the Stack Composes 15

9.2 What Is Specific to This Work 16

9.3 Limitations 16

10. Related Work 16

11. Conclusion 17

Appendix A. Implementation Map 17

Appendix B. Reproducibility Notes 18

References 18

Abstract

We present Corsaire-1, a supervised fine-tuning run and training system for a 50%-expert-pruned DeepSeek-V3.2 mixture-of-experts model under an end-to-end low-bit numerical regime. The model starts from a REAP-pruned DeepSeek-V3.2 checkpoint with 128 routed experts, adds lightweight gated rescaling in the normalization and attention-output paths, and is fine-tuned on the `BlaiseAI/blaise-sft-training-mix` conversational data mix. The training target keeps weights, activations, and KV-cache paths at four bits or below wherever the hardware path permits, stores FlashAdamW optimizer moments in compressed integer state, and removes the persistent high-precision master-weight copy.

The central contribution is not a new base architecture. The contribution is a composition of numerical and systems techniques that make this compressed model trainable rather than merely loadable: stochastic NVFP4 update casts, FlashAdamW with weight-side ECO residual feedback, a transient-master update path, Activation Residual Compensation (ARC) for NVFP4 input-cast bias in weight gradients, TurboQuant for differentiable 2.5-bit MLA latent storage, IndexCache-style FP8 fake quantization for DSA indexer keys, GatedNorm, G1 attention-output gates, StreamBP sequence replay, zero-cost checkpointing, and B200-oriented CUDA/Triton/CuTe kernel work. We give the mathematical contracts for the low-bit operators, the optimizer-state invariants, the distributed training layout, and the validation evidence available at this stage. The result is a 16xB200 SFT recipe for training a roughly 202B-parameter Megatron checkpoint with a public 345B-parameter REAP lineage and W4A4KV4/IndexerK8-FP8 deployment target, while preserving the sparse-attention and MoE behavior of the source model.

1. Introduction

Large sparse mixture-of-experts (SMoE) language models are attractive because they separate total parameter count from activated parameter count, but that same total-parameter footprint dominates training memory. This is especially visible in the DeepSeek-V3 family, where Multi-Latent Attention (MLA) compresses the dense KV cache and the MoE block limits per-token compute, yet the model still carries hundreds of billions of stored parameters and large optimizer state. Sparse attention in DeepSeek-V3.2 further reduces long-context attention compute by selecting a top-k subset of keys per query, but it introduces a new indexer path whose cache and training gradients also need to be made efficient.

Corsaire-1 is built around a pragmatic question: can a high-quality, pruned DeepSeek-V3.2 MoE be fine-tuned under the same low-bit constraints that make it deployable? Post-training quantization alone does not answer this question. Quantized inference can keep a model compact at rest and at decode time, while training usually reintroduces a high-precision master copy of the parameters, full-precision optimizer moments, high-precision saved activations, and full-precision fake-quant reference paths. Those hidden training buffers can erase much of the memory benefit of the quantized checkpoint.

The Corsaire-1 stack therefore treats compression as a training-system contract. Every component has to state where rounding error appears, where the missing information is carried, and how the backward pass remains meaningful. Weight quantization error is fed into Adam's first moment through ECO. Activation rounding error is projected directly into the weight gradient through ARC. TurboQuant and IndexCache do not stop at forward-only cache compression: they include analytic backwards matched against differentiable straight-through oracles. StreamBP is not a generic recompute flag: it replays the long sequence in causal chunks while respecting DSA, MoE, activation hooks, and DDP bucket readiness. Zero-cost checkpointing snapshots the actual compressed restart state rather than forcing the system back through uncompressed checkpoint formats.

This report is written as a publication draft rather than a commit-history summary. The branch history was used only to reconstruct the technical content of the run. We emphasize the pieces that are specific to the BlaiseAI training stack and cite the source methods that the stack builds upon.

Contributions

1. **End-to-end compressed SFT recipe.** We specify a DeepSeek-V3.2-REAP SFT target with NVFP4 W4A4 paths, KV4 deployment target, TurboQuant on the dense MLA latent, FP8 IndexerK8 on DSA indexer keys, GatedNorm, G1 attention gates, and FlashAdamW compressed optimizer state.
2. **Master-weight-free NVFP4 optimizer path.** We integrate FlashAdamW with ECO so the persistent FP32/BF16 master copy is removed. Each update uses a transient high-precision shard, casts immediately back to NVFP4, injects the cast residual into the first moment, and frees the transient shard.
3. **Activation Residual Compensation (ARC).** We introduce an activation-side correction for NVFP4 input casts. ARC is not optimizer ECO: it compensates the bias in dW caused by using $q(x)$ in the forward matmul by injecting $dy^T (x - q(x))$ into the weight gradient at the same linear site.
4. **Trainable cache quantizers.** TurboQuant and IndexCache are wired as differentiable fake-quant operators with analytic backward kernels, making 2.5-bit MLA-latent and FP8 DSA-indexer-key compression available during training rather than only inference.
5. **Long-context systems integration.** We combine StreamBP replay, DSA Triton kernels, compact top-k indices, optional BF16 K/V gradient atomics, B200 CuTe kernels, NCCLX/TorchComms transport work, and zero-cost checkpointing into a coherent 16xB200 Megatron training path.

2. Base Model, Data, and Training Target

2.1 Model Lineage

The base model family is DeepSeek-V3.2, which extends the DeepSeek-V2/V3 line with MLA, SMoE feed-forward blocks, long-context positional scaling, and DeepSeek Sparse Attention (DSA). MLA compresses each token's KV state into a low-dimensional latent, while DSA adds a lightweight indexer that chooses the top-k keys consumed by sparse attention. These components are inherited from the base model; Corsaire-1 focuses on training them under an aggressive low-bit budget.

The checkpoint used here is derived from a REAP-pruned DeepSeek-V3.2 model:

BlaiseAI/DeepSeek-V3.2-REAP-345B-NVFP4-W4A4KV4-IndexerK8-FP8-GatedNorm-G1. The REAP lineage prunes routed experts by combining router gate values and expert activation norms. In the published REAP DeepSeek-V3.2 variant, the model is compressed from 256 routed experts to 128 routed experts, retains top-8 routing, and keeps the 163,840-token context target. Our training stack uses the same 128-expert post-prune structure and re-equilibrates routing with the standard expert-bias update during SFT.

The effective public naming is slightly subtle. The REAP lineage is commonly identified as 345B because it is the 50%-expert-pruned descendant of the larger DeepSeek-V3.2 MoE. The trainable checkpoint in this stack has roughly 202B stored parameters after quantized and architectural conversion details are accounted for in Megatron. In this report, "345B lineage" refers to the public REAP model family; "202B trainable checkpoint" refers to the Megatron-side training object.

2.2 Architecture Snapshot

The SFT configuration keeps the DeepSeek-V3.2 decoder shape:

Component	Value
Decoder layers	61
Hidden size	7168
Attention heads	128
QK head dim	128
RoPE positional dim	64
V head dim	128
Dense FFN hidden size	18432
MLA KV latent rank	512
MLA query latent rank	1536
DSA indexer heads	64
DSA indexer head dim	128
DSA selected keys per query	2048
MoE routed experts	128
Routed experts per token	8
Shared expert intermediate size	2048
Routed expert intermediate size	2048
Maximum position	163840
SFT sequence length	32768

The first three layers are dense FFN layers, and layers 4 through 61 use the MoE FFN. Router scoring uses the sigmoid pre-softmax routing path with group top-k over 8 groups and 4 selected groups. Router bias is enabled with update rate $1e-3$, router auxiliary loss coefficient $1e-4$, and top-k scaling factor 2.5.

2.3 Data

The SFT data source is `BlaiseAI/blaise-sft-training-mix`, prepared through the `nemotron-full-family` configuration in the repo tooling. The target mixture is approximately 1B DeepSeek-tokenizer tokens of conversational SFT data. The records are normalized into an OpenAI-style message list with `system`, `user`, `assistant`, `tool`, and `developer` roles when present. Empty assistant messages followed by tool outputs are converted into explicit assistant tool calls so the DeepSeek-V3.2 tokenizer and chat template see a stable tool-use protocol.

The training loss is assistant-token-only cross entropy. System, user, tool output, and template-control regions are masked with the ignore index. This is important for an agentic/tool corpus because the model should learn assistant responses and tool-call syntax without being trained to reproduce user prompts or tool observations.

The production launcher includes a double-pass option over the nominal 1B-token target: 61056 packed samples at sequence length 32768 correspond to about 2.0B token slots before assistant-only masking. We treat the 1B-token mix as the dataset definition and the 2B-token setting as a run-length knob rather than a distinct dataset.

3. Numerical Design

3.1 NVFP4 W4A4 as the Shared Compute Substrate

NVFP4 stores values as FP4 E2M1 codes with a local FP8 E4M3 scale shared by a block of 16 elements and an additional higher-level scale used by the software recipe. NVIDIA's Transformer Engine exposes this format through the NVFP4BlockScaling recipe on Blackwell hardware. The format is attractive because the block size is small enough to reduce outlier damage relative to coarser microscaling formats, but it is still lossy enough that repeated round-to-nearest casts are dangerous during training.

For model-parameter update casts, Corsaire-1 uses stochastic rounding. Given a scaled real value z lying between adjacent FP4 grid points $a \leq z \leq b$, the cast emits a with probability $(b - z) / (b - a)$ and b with probability $(z - a) / (b - a)$. Therefore $E[q_{sr}(z)] = z$ on the local grid. In practice the implementation uses a block dither kernel with a size-threshold autotuned path: small shards use the fixed launch configuration to avoid autotune overhead, while large shards use autotuned launch parameters that reduce wall time on 16M+ element tensors.

Forward activation casts remain owned by Transformer Engine's NVFP4 linear path. We do not replace TE's GEMM or its internal activation cast. Instead, ARC corrects the weight-gradient bias produced by that cast, as described in Section 5.

3.2 SpinQuant Rotations

Low-bit block scaling is sensitive to channel outliers. SpinQuant-style orthogonal rotations reduce this sensitivity by spreading energy more evenly across channels before quantization. Algebraically, if R is orthonormal then $(x R)(R^T W) = x W$ in exact arithmetic. Under a nonlinear quantizer the identity is no longer exact, but the rotated distribution typically has lower per-block dynamic-range pressure.

The implementation supports deterministic random signed Hadamard rotations and loaded rotation matrices. $R1$ is a global hidden-state rotation. $R2$ is a per-layer, per-head rotation used on attention value/KV-sensitive paths. Rotations are constructed from deterministic seeds so TP, SP, CP, and EP ranks produce identical buffers without communication. The SFT recipe enables SpinQuant in W4A4K4V4 mode; weight fusion can be enabled for conversion or validation runs when an offline-fused checkpoint is desired.

3.3 TurboQuant for the Dense MLA Latent

The dense MLA latent c_KV has dimension 512 and is stored once per token per layer. It is the largest dense KV-side object in the model. TurboQuant reduces this latent to an average of 2.5 bits per coordinate with a block-Hadamard codebook while retaining a differentiable fake-quant path for training.

For each token latent x in R^{512} :

```
u = x / ||x||_2
v = H(u * s1) * s2
w = Q_2.5(v)
x_hat = ||x||_2 * norm_correction * (H(w * s2) * s1)
```

Code block: text

Here H is the normalized Walsh-Hadamard transform, $s1$ and $s2$ are frozen layer-scoped sign vectors, and $Q_{2.5}$ is a two-tier Lloyd-Max scalar quantizer: in each group of 128 channels, 32 channels use a 3-bit codebook and 96 channels use a 2-bit codebook. The resulting average rate is:

$$(32 * 3 + 96 * 2) / 128 = 2.5 \text{ bits / coordinate.}$$

Code block: text

The backward pass uses an analytic straight-through estimator with saturation masks and the derivative of the normalization step. The CUDA backward kernel was added specifically for training; the reference SGLang-style path was forward-only. The operator is per-token and acts only on the final latent dimension, so it commutes with sequence and context sharding. In the Megatron path, the hook sits after `kv_layernorm` and before the MLA up-projections, where each rank has the full 512-dimensional latent.

Validation in the repo covers deterministic buffer construction, forward parity against the reference math, analytic backward parity against a differentiable STE oracle, CUDA forward/backward parity, and two-rank parallelism checks.

3.4 IndexCache-Style FP8 DSA Indexer Key Quantization

DSA uses an indexer key tensor of dimension 128 after a Hadamard rotation. IndexCache, as a paper, focuses on cross-layer reuse of top-k index selections. Corsaire-1 adopts a narrower but immediately useful piece for training: FP8 E4M3 fake quantization of the post-rotation DSA indexer key. Cross-layer top-k reuse remains a follow-up because it requires decoder-loop changes to thread selected indices across layers.

For a row x in R^{128} , the fake quantizer computes:

```
a = max(max_j |x_j|, eps)
s = a / 448
n_j = cast_fp8_e4m3(clamp(x_j / s, -448, 448))
x_hat_j = s * n_j
```

Code block: text

The backward has two terms. The first is the standard STE on non-saturated coordinates. The second is a rank-1 scale-path correction on the coordinate that produced the row maximum:

```
d x_j =
  d y_j * clip_mask_j
  + 1[j = argmax |x|] * sign(x_j) * eps_active / 448
    * sum_i d y_i * (n_i - clip_mask_i * x_i / s)
```

Code block: text

This rank-1 term is required because s depends on the row maximum. A previous bug omitted the $-clip_mask_i * x_i / s$ piece; the branch history records that fix explicitly, and the current test suite compares the analytic backward against an STE-detach autograd oracle to fp64 tolerance.

The operator is applied to indexer K only. Indexer Q remains unquantized because it is recomputed per query and does not create a persistent cache footprint.

4. Architectural Additions

4.1 GatedNorm

GatedNorm inserts a low-rank learned gate after RMSNorm. Given the RMS-normalized input y and rank $r = 16$:

```
z = y W_down^T
a = SiLU(z)
```



```
g = sigmoid(a W_up^T)
out = y * g
```

Code block: text

The parameter cost per normalization site is $2 * \text{hidden_size} * r$, which is small relative to the MoE weights. The implementation avoids materializing the full-width gate in memory. It saves only the rank-sized z activation and computes the down projection, SiLU, up projection, sigmoid, and multiply in a fused path. A CuTe SM100 forward kernel is available on B200, with Triton and torch-MM fallbacks selected by shape and environment knobs. The backward formulas match the reference:

```
dlogits = dout * y * g * (1 - g)
dW_up   = dlogits^T @ SiLU(z)
da       = dlogits @ W_up
dz       = da * SiLU'(z)
dW_down = dz^T @ y
dy       = dout * g + dz @ W_down
```

Code block: text

The motivation is twofold. First, the GatedNorm paper frames residual outliers as part of an outlier-driven rescaling mechanism around normalization. A lightweight explicit gate can retain the useful rescaling behavior with fewer extreme activations. Second, smoother post-normalization activations are valuable in W4A4 training because they reduce the load placed on per-block scales.

4.2 G1 Attention Output Gate

The attention gate follows the G1 position from the gated-attention study: after scaled-dot-product attention and before the output projection. For a head-wise attention output o and query-derived gate γ :

```
gamma = sigmoid(W_g q)
out = W_o (o * gamma)
```

Code block: text

In standard self-attention this is the SDPA-output gate. In MLA/DSA the logical placement is still between the per-head attention result and the final output projection, after the latent has been expanded into per-head values. The fork contains both Python and fused CUDA paths for the gate. The B200 CuTe port uses fast sigmoid and adaptive launch geometry; backward stays analytic and is validated against the Megatron reference.

We include G1 because it is cheap, improves robustness in the source literature, and gives the W4A4 stack another learned rescaling point at a numerically sensitive boundary.

5. Optimizer and Error Feedback

5.1 FlashAdamW State Compression

FlashAdamW is used as the optimizer. Its first and second moments are stored in compressed grouped integer state with floating scales rather than persistent FP32 tensors. In this fork, compressed state dict support preserves the quantized moment payloads and scales on checkpoint save instead of dequantizing them. DistributedOptimizer is extended so compressed state wrappers can be allocated on load without briefly materializing full optimizer state.

The SFT recipe uses AdamW betas $(0.9, 0.95)$, zero weight decay, gradient clip 1.0 , cosine decay from $5e-6$ to $1e-7$, and FlashAdamW ECO enabled.

5.2 Weight-Side ECO and the Transient Master

Let θ_t be the high-precision parameter value that a conventional master-weight optimizer would hold, and let $q(\theta_t)$ be the NVFP4 model parameter written back after the update. The missing information is the cast residual:

$$e_t = \theta_t - q(\theta_t)$$

Code block: text

ECO injects this residual into Adam's first moment:

$$\begin{aligned} m_t &\leftarrow m_t + \alpha_t * D_t * e_t \\ D_t &= \sqrt{v_{\text{hat}}_t} + \epsilon \\ \alpha_t &= ((1 - \beta_1^t) / lr) * (1 - 1 / \beta_1) \end{aligned}$$

Code block: text

The injection is per-step and per-element. It cannot be delayed across multiple steps without changing the optimizer trajectory, because the residual is tied to the exact post-update cast.

The important engineering change in Corsaire-1 is that there is no persistent master shard for NVFP4 parameters. At step time, FlashAdamW dequantizes the local NVFP4 shard into a transient BF16/FP32 update buffer, applies AdamW, casts the result back to NVFP4 with stochastic rounding, injects the pre-cast/post-cast residual into the moment, and deletes the transient buffer. DistributedOptimizer keys optimizer state by the stable NVFP4 model parameter and attaches shard-offset metadata so FlashAdamW can reconstruct only the local transient shard.

This makes ECO a correctness mechanism, not an optional enhancement. Without ECO, removing the persistent master leaks the residual every step and introduces a drift term that grows with the number of updates.

5.3 Activation Residual Compensation (ARC)

ARC is the activation-side companion to the optimizer ECO path, but it is not itself optimizer ECO. Activations have no persistent state into which an optimizer can inject error. The relevant missing information is instead the weight-gradient contribution lost when a linear layer's forward path consumes $q(x)$ rather than x .

For a linear layer:

$$\begin{aligned} y &= q(x) W^T \\ dw_{\text{naive}} &= dy^T q(x) \\ dw_{\text{bf16}} &= dy^T x \end{aligned}$$

Code block: text

The rounding-induced weight-gradient bias is:

$$dw_{\text{naive}} - dw_{\text{bf16}} = -dy^T (x - q(x))$$

Code block: text

ARC adds the residual projection:

$$dw_{\text{arc}} = dw_{\text{naive}} + dy^T (x - q(x)) = dy^T x$$

Code block: text

Thus ARC recovers the BF16 weight gradient for the linear layer while leaving the activation gradient on the standard saturated-zero STE path. The implementation attaches hooks to Transformer Engine linears: a forward pre-hook captures or recomputes the input, a backward tap observes dy , and the weight gradient hook adds the correction. In production, activation recomputation or StreamBP replay supplies the needed input without saving another full BF16 copy for every layer.

The CPU correctness suite verifies forward finiteness, range envelopes, STE dx parity, exact dW equality with the BF16 reference, and statistical bias reduction across random trials.

5.4 Gradient Release and Bucket-Completion Overlap

Gradient release steps parameters as soon as their reduced gradients are ready, then frees the gradient buffer. This can reduce parameter-associated peak memory from full-model gradients to one parameter or one bucket of gradients.

For plain FlashAdamW, per-parameter gradient release is direct. For NVFP4 + ECO, the per-parameter step only produces the transient updated shard; the TE NVFP4 cast and ECO injection normally happen later in the distributed optimizer copy-back phase. The fork therefore adds an `NVFP4EcoGradientReleaseOrchestrator` that buffers completed NVFP4 parameters and fires the cast plus `inject_eco_error` once the expected set is ready.

For Megatron-Core DDP with overlapped reduce-scatter, the implementation hooks bucket completion. Each bucket records an event after reduce-scatter, runs the per-parameter optimizer step on a high-priority optimizer stream, flushes the NVFP4 cast/ECO injection for that bucket, and zeros the bucket gradient data. The scheduler has guards for CUDA graph capture, re-entry, missing gradients, exception propagation, and StreamBP chunking.

5.5 Autotune Correctness

The ECO inject kernel and the stochastic dither kernel both perform in-place read-modify-write operations on live optimizer state or update buffers. Triton autotune benchmarks candidate launch configurations by executing the kernel. If the autotuner is not told how to restore mutated inputs, the first call for a new shape silently applies the in-place operation once per candidate plus once for the selected configuration.

The current implementation marks the mutated tensors with `restore_value`, and the test suite includes a first-call regression test that invokes the same prepared state twice without cloning. This is a small implementation detail, but it is essential for scientific reproducibility: a numerically correct kernel can still corrupt training if autotune mutates state during benchmarking.

6. Long-Context Systems Work

6.1 StreamBP

StreamBP is a native Megatron replay path for long decoder sequences. The forward pass runs without storing the full layer graph. During backward, layers are replayed over query chunks `[i : j]` while keys and values cover the causal prefix `[ : j]`. This preserves decoder semantics and reduces activation memory.

The implementation supports decoder self-attention, TP, PP, DP, sequence parallelism when CP is disabled, DSA-backed attention, MoE layers, activation hooks, and LM-head loss chunking. MoE layers use exact full-layer replay under the same

wrapper because expert parameters may receive tokens in only a subset of chunks; per-chunk DDP readiness would otherwise step sparse expert weights too early. For the DeepSeek-V3.2 SFT target, StreamBP runs with CP disabled and typical chunk size 2048.

Local production validation on H200 showed dense StreamBP at 0.6065x peak memory and 1.0898x throughput relative to full activation recompute for the tested dense configuration. Mixed dense/MoE validation showed 0.5192x peak memory and 1.0190x throughput, with loss parity and bounded `main_grad` differences. These are subsystem validations, not final Corsaire-1 quality numbers.

6.2 DSA Kernels and Sparse-Attention Bottlenecks

The DSA path has three expensive pieces: indexer scoring, top-k selection, and sparse attention forward/backward over selected edges. Profiling on B200 shows that sparse DSA backward is the dominant GPU kernel in full-model probes. One profile attributed roughly 70.9% of aggregate GPU kernel time to `_sparse_dsa_backward_kernel`, 11.9% to `_sparse_dsa_forward_kernel`, and 9.0% to `_dsa_indexer_scores_kernel`, with NCCL send/recv second-order.

The current stack therefore focuses on preserving exact DSA semantics while reducing memory traffic and launch overhead:

- Triton DSA forward/backward kernels are enabled for the SFT path.
- Streaming indexer top-k avoids materializing the full score tensor when the indexer loss is disabled.
- Compact top-k indices can store selected indices as int16 when sequence length permits, reducing memory traffic for top-k buffers.
- The SFT script exposes BF16 K/V gradient atomics as a performance knob. FP32 atomics remain the strict-control path; BF16 atomics are used when the observed error stays within the run's tolerance.
- DSA score/top-k workspace reuse reduces allocator churn during StreamBP replay.

The most important open kernel target is DSA backward. A key-tile or selected-edge-reduction design could reduce random atomics and improve K/V reuse without changing selected indices, but it requires a deeper CUDA rewrite than the current Triton path.

6.3 B200 CuTe Kernel Ports

The `flashtesting-kernels` worktree ports selected optimized SM100 kernels from `optimization-playground` into the Megatron fork:

Kernel	Change	Validation	Observed effect
G1 gate	Fast sigmoid, adaptive launch geometry	0.0 max diff forward vs baseline	1.22x-1.43x vs paper baseline at small N
GatedNorm	CuTe SM100 forward with tensor cores, padded SMEM, fallback guards	<= 1e-3 max abs vs Triton	1.48x-1.81x vs torch-MM on measured shapes
IndexCache	Dead-store/scratch cleanup in reductions	bit-exact outputs	launch-overhead dominated; no semantic change

The TurboQuant rope-copy optimization from the upstream playground does not apply because the Megatron TurboQuant kernel only processes the 512-dimensional MLA latent; RoPE is a separate tensor in this path.

6.4 Zero-Cost Checkpointing

Zero-cost checkpointing (ZCC) snapshots the stable post-update parameter and optimizer state to host memory without forcing the normal synchronous distributed checkpoint path. It records model parameters, quantized FlashAdamW state, error-correction state, persistent parameter-side quantizer tensors, scheduler state, RNG state, and the NVFP4 stochastic-rounding dither counter when enabled. Transient fields such as `_fa_updated_shard` are intentionally excluded.

FlashAdamW optimizer states are fused into a contiguous uint8 CUDA buffer so ZCC can copy one large region rather than thousands of small tensors. The flash tier writes atomically with checksums; a durable tier can be written by background workers. Recovery validates checksums, restores tensors into the live optimizer layout, and falls back from flash to durable when needed.

ZCC is not claimed as a replacement for cold-start distributed checkpointing. It is a hot-restart and failure-containment path for long low-bit runs where the compressed optimizer state is the real restart state.

7. Training Configuration

The primary production shape is a 2-node, 16xB200 run:

Setting	Value
GPUs	16 x B200
Tensor parallel	4
Pipeline parallel	4 for memory-constrained probes; 2 supported by script
Context parallel	1
Expert parallel	4
Expert tensor parallel	1
Sequence parallel	enabled
Sequence length	32768
Micro batch	2 in tmutex production launcher
Global batch	32 in tmutex production launcher
DSA top-k	2048
StreamBP chunk	2048
DSA chunk	2048 or 4096 depending on probe
Optimizer	FlashAdamW + ECO
LR	5e-6 cosine to 1e-7
Warmup samples	31348
Gradient clip	1.0
Weight decay	0.0
Precision	BF16 compute with NVFP4 FP4 recipe
Quantization features	SpinQuant, TurboQuant, IndexCache
Architectural gates	GatedNorm rank 16, G1 attention output gate

The launcher supports standard NCCL and an NCCLX/TorchComms transport path. The NCCLX branch adds runtime feature gates, RoCE/RDMA profile selection, memory pool support, and topology checks. Because the current bottleneck is DSA compute rather than communication, NCCLX is treated as a transport optimization rather than part of the numerical recipe.

8. Validation

8.1 Correctness Surface

The low-bit stack is validated at three levels: local operator math, distributed parallelism, and end-to-end training probes.

Component	Validation evidence
TurboQuant	Buffer reproducibility; forward shape/finiteness; quantization-error bounds; FWHT self-inverse; analytic backward equals STE autograd oracle; CUDA forward/backward parity; TP/SP/CP/EP parallelism tests
IndexCache	FP8 row fake-quant shape/finiteness; zero-row eps clamp; analytic backward equals STE autograd oracle; dtype matrix; SGLang act-quant math parity when FP8 is available
ARC	NVFP4 block range envelope; quant-error bounds; exact dW match to BF16 reference; STE dx parity; statistical bias reduction over 200 trials; finite saturated-lane gradients
GatedNorm	Fused forward/backward parity with torch reference; transformer integration tests; rank-aware torch-MM fallback; CUDA path smoke
G1 gate	Fused forward and backward tests; B200 kernel diff checks
FlashAdamW + ECO	FP64 reference checks; quantized/unquantized paths; first-call autotune restore regression; end-to-end FlashAdamW ECO tests
Gradient release	50-step parity checks; MCore DDP support; bucket-overlap ordering; exception and graph-capture guards; StreamBP compatibility checks
StreamBP	Loss/main-grad parity against recompute baselines; dense and MoE replay checks; DSA and activation-hook compatibility
ZCC	Snapshot/load/restore; compressed state fusion; durable fallback; 8-rank stress with deliberate flash corruption

8.2 Microbenchmarks

TurboQuant forward plus backward on B200, latent dim 512:

Tokens	Forward	Backward	Total	Tokens/sec
256	38 us	156 us	194 us	1.3M
1024	38 us	158 us	195 us	5.3M
4096	38 us	159 us	197 us	20.8M
16384	99 us	131 us	231 us	71.1M
65536	391 us	500 us	891 us	73.6M

ECO inject kernel on the quantized optimizer-state path:

Elements	Baseline	Tuned + restore	Effect
262144	35 us	47 us	small-shard overhead
4194304	36 us	48 us	small-shard overhead
16777216	123 us	52 us	2.36x faster
67108864	550 us	176 us	3.13x faster

NVFP4 stochastic dither threshold dispatch:

Elements	Path	Time	Effect
262144	fixed small-shard	25 us	near original
4194304	fixed small-shard	24 us	near original
16777216	autotuned large-shard	33 us	about 60% faster
67108864	autotuned large-shard	86 us	about 73% faster

The large-shard wins matter because FFN matrices, embeddings, and large expert shards live in the 10M-100M element regime. Small expert or LoRA-like shards stay on the fixed path.

8.3 Short-Run Training Probes

The full 16xB200 stack has been exercised in short probes with DSA, FlashAdamW+ECO, SpinQuant, TurboQuant, IndexCache, GatedNorm, G1 gates, StreamBP, and distributed optimizer overlap enabled at sequence length 32768. Two sanity probes are important:

- **LR = 0 probe.** Multiple SFT iterations complete with no NaN/Inf in forward, backward, or optimizer step. Loss behavior is consistent with zero learning rate.
- **LR = 5e-6 probe.** Multiple SFT iterations complete and the loss decreases, demonstrating that gradients survive the full low-bit path and sparse MoE bucket accounting.

This draft should not be read as a final benchmark paper for the trained model. The public version is expected to add full 1B-token SFT curves and task evaluations. The contribution documented here is the training recipe and the validation of the low-bit path that makes the full run feasible.

9. Discussion

9.1 Why the Stack Composes

Most failures in low-bit training are not caused by one catastrophic rounding event. They are caused by small biased residuals that are applied repeatedly: the parameter residual lost after every update cast, the activation residual lost in every weight-gradient matmul, the scale-path residual omitted from a fake-quant backward, or the state mutation hidden inside an autotune sweep. Corsaire-1's design principle is to carry each residual at the place where it matters:

- Weight update residuals are optimizer-state residuals, so ECO injects them into Adam's first moment.
- Activation input residuals are weight-gradient residuals, so ARC injects them into `dW`.
- Row-scale fake-quant residuals affect the max coordinate, so IndexCache adds a rank-1 gradient correction.
- Latent normalization residuals affect every coordinate, so TurboQuant includes the normalization derivative in the backward.
- Autotune residuals are implementation state mutations, so kernels declare `restore_value`.

This is why the stack can tolerate multiple aggressive quantizers at once. Each one is allowed to be lossy, but none is allowed to hide where its lost information went.

9.2 What Is Specific to This Work

The base model, MLA, DSA, REAP, gated attention, GatedNorm, NVFP4, stochastic rounding, and ECO each have their own prior work. The BlaiseAI-specific work is the composition and productionization:

- Converting a compressed NVFP4 Hugging Face checkpoint into a native Megatron training checkpoint with DSA, G1, and GatedNorm mappings.
- Making forward-only cache quantization kernels differentiable and parallelism safe.
- Removing the persistent master from an NVFP4 distributed optimizer while retaining ECO semantics.
- Adding ARC so activation quantization has a weight-gradient correction instead of relying on hope or saved high-precision activations.
- Making StreamBP, DSA, MoE, activation hooks, and gradient release agree on when gradients are complete.
- Preserving compressed optimizer state through checkpointing and restart.

9.3 Limitations

The strongest limitation is that the current validation is a training-system validation plus short-run stability evidence. A final public release is expected to include full training curves over the 1B-token mix, evaluation on tool-use, coding, reasoning, long-context, and regression tasks, and ablations that remove ARC, TurboQuant, IndexCache, GatedNorm, and G1 individually.

The DSA implementation is still a first-order performance bottleneck. Current optimizations reduce overhead without changing DSA semantics, but the backward kernel remains atomics-heavy. A deeper selected-edge or key-tiled backward is likely needed for the next major speed step.

The activation-side correction assumes the TE cast observed by ARC's reference quantizer matches the production NVFP4 cast closely enough. CPU tests prove the mathematical correction; publication validation is expected to include a direct TE-cast-vs-ARC-cast parity test on the production Transformer Engine build.

The 163,840-token maximum position is inherited from the model, but the SFT run described here is validated at 32,768 tokens. Extrapolation under compressed DSA indexer keys remains an evaluation item.

10. Related Work

DeepSeek MLA, MoE, and DSA. DeepSeek-V2 introduced MLA and DeepSeekMoE as a way to reduce KV cache and training cost in a large MoE. DeepSeek-V3 extended the line with larger-scale MoE training and FP8-oriented engineering, and DeepSeek-V3.2 introduced DSA for long-context sparse attention. Corsaire-1 uses this architecture as the substrate and focuses on compressed training.

REAP expert pruning. REAP argues for one-shot expert pruning over expert merging for generative tasks and scores experts using router activity and expert activation norms. Corsaire-1 starts from a 50%-expert-pruned REAP lineage and fine-tunes it under low-bit constraints.

Rotation-aware quantization. SpinQuant and related GPTQ/QuIP-style work show that rotations can improve low-bit quantization by reducing outlier concentration. Corsaire-1 uses deterministic SpinQuant-style rotations as part of an online W4A4 training recipe.

NVFP4 training. NVIDIA's NVFP4 and Transformer Engine recipes provide the hardware/software substrate for FP4 block-scaled computation on Blackwell. Corsaire-1 adds optimizer residual feedback, ARC, and trainable cache quantizers around that substrate.

Memory-efficient optimizers. FlashOptim/FlashAdamW compress optimizer state and reduce parameter-associated training memory. ECO removes persistent master weights by injecting quantization residuals into momentum. Corsaire-1 combines these ideas with NVFP4 transient masters, distributed optimizer sharding, and gradient-release bucket overlap.

Gating and outlier rescaling. Gated attention and GatedNorm motivate explicit learned rescaling as a way to improve training stability, reduce sink pathologies, and improve quantization robustness. Corsaire-1 uses both in the low-bit SFT target.

IO-aware and sparse attention systems. FlashAttention established the value of IO-aware attention kernels. DSA and IndexCache target sparse long-context attention. Corsaire-1 keeps DSA exact at the selected-index level while compressing the indexer-key cache and reducing replay overhead.

11. Conclusion

Corsaire-1 is a compressed-training recipe for a 50%-expert-pruned DeepSeek-V3.2 MoE, not just a quantized checkpoint. The system removes the persistent master-weight copy, keeps optimizer states compressed, trains through 2.5-bit and FP8 cache fake-quants, compensates both parameter and activation cast residuals, and integrates the result into a long-context Megatron training path on 16xB200.

The core lesson is that low-bit training needs explicit residual accounting. Once every lossy boundary has a defined backward or feedback path, the pieces compose: NVFP4 update casts, ARC-corrected activation casts, TurboQuant, IndexCache, GatedNorm, G1 gates, FlashAdamW, StreamBP, and DSA can operate in the same run without falling back to a full-precision training regime.

Appendix A. Implementation Map

Area	Primary files
DeepSeek SFT launch	examples/sft/run_sft_deepseek_nvfp4.sh, examples/sft/launch_sft_deepseek_nvfp4_tmux.sh
Dataset preparation	tools/prepare_blaise_sft.py, examples/sft/prepare_data.sh
Checkpoint conversion	tools/convert_blaise_deepseek_v32_reap_to_megatron.py
TurboQuant	megatron/core/quantization/turboquant/
IndexCache fake quant	megatron/core/quantization/indexcache/
ARC / NVFP4 activation correction	megatron/core/quantization/nvfp4_act_eco/
SpinQuant	megatron/core/quantization/spinquant.py
FlashAdamW + ECO	megatron/core/optimizer/flash_optimizers.py
NVFP4 stochastic rounding	megatron/core/optimizer/nvfp4_sr.py
Distributed optimizer integration	megatron/core/optimizer/distrib_optimizer.py
StreamBP	megatron/core/transformer/streambp.py
DSA	megatron/core/transformer/experimental_attention_variant/dsa.py, dsa_triton.py
GatedNorm	megatron/core/fusions/gated_norm.py, gated_norm_cute.cu
G1 gate	megatron/core/fusions/fused_g1_gate.py, fused_g1_gate.cu
ZCC	megatron/core/optimizer/zero_cost_checkpoint/
NCCLX/TorchComms	megatron/core/torchcomms_adapter.py, megatron/core/distributed/ncclx_*.py

Appendix B. Reproducibility Notes

The current branch is `flashtraining`. The report was reconstructed from the branch history since commit `f8c97048e48521b48902a733d0e3f1f642edb5fc` and from related non-main branches: `flashoptim`, `turboquant-indexcache`, `flashoptim-gradientrelease`, `flashoptim-zcc`, `flashoptim-ncclx`, `flashoptim-streambp`, and `flashoptim-kernels`. The main branch was not used as a source of technical content.

The user worktree had an uncommitted launch-script edit and an untracked `blaise-docs/draft.md` before this report was created. This report is added as a separate source file so the original draft remains available.

References

- [1] DeepSeek-AI et al. "DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model." arXiv:2405.04434, 2024. <https://arxiv.org/abs/2405.04434>
- [2] DeepSeek-AI et al. "DeepSeek-V3 Technical Report." arXiv:2412.19437, 2024. <https://arxiv.org/abs/2412.19437>
- [3] DeepSeek-AI et al. "DeepSeek-V3.2: Pushing the Frontier of Open Large Language Models." arXiv:2512.02556, 2025. <https://arxiv.org/abs/2512.02556>
- [4] Mike Lasby, Ivan Lazarevich, Nish Sinnadurai, Sean Lie, Yani Ioannou, and Vithursan Thangarasa. "REAP the Experts: Why Pruning Prevails for One-Shot MoE compression." arXiv:2510.13999, 2025. <https://arxiv.org/abs/2510.13999>
- [5] Zihan Qiu et al. "Gated Attention for Large Language Models: Non-linearity, Sparsity, and Attention-Sink-Free." arXiv:2505.06708, 2025. <https://arxiv.org/abs/2505.06708>

- [6] Zihan Qiu et al. "A Unified View of Attention and Residual Sinks: Outlier-Driven Rescaling is Essential for Transformer Training." arXiv:2601.22966, 2026. <https://arxiv.org/abs/2601.22966>
- [7] Zechun Liu et al. "SpinQuant: LLM quantization with learned rotations." arXiv:2405.16406, 2024. <https://arxiv.org/abs/2405.16406>
- [8] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. "GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers." arXiv:2210.17323, 2022. <https://arxiv.org/abs/2210.17323>
- [9] Jerry Chee et al. "QuIP: 2-Bit Quantization of Large Language Models With Guarantees." arXiv:2307.13304, 2023. <https://arxiv.org/abs/2307.13304>
- [10] NVIDIA. "NVFP4." Transformer Engine documentation, 2026. https://docs.nvidia.com/deeplearning/transformer-engine/user-guide/features/low_precision_training/nvfp4/nvfp4.html
- [11] NVIDIA. "Introducing NVFP4 for Efficient and Accurate Low-Precision Inference." NVIDIA Technical Blog, 2025. <https://developer.nvidia.com/blog/introducing-nvfp4-for-efficient-and-accurate-low-precision-inference/>
- [12] Suyog Gupta, Ankur Agrawal, Kailash Gopalakrishnan, and Prithish Narayanan. "Deep Learning with Limited Numerical Precision." ICML 2015. <https://arxiv.org/abs/1502.02551>
- [13] Mahdi Nikdan, Amir Zandieh, Dan Alistarh, and Vahab Mirrokni. "ECO: Quantized Training without Full-Precision Master Weights." arXiv:2601.22101, 2026. <https://arxiv.org/abs/2601.22101>
- [14] Jose Javier Gonzalez Ortiz, Abhay Gupta, Chris Renard, and Davis Blalock. "FlashOptim: Optimizers for Memory Efficient Training." arXiv:2602.23349, 2026. <https://arxiv.org/abs/2602.23349>
- [15] Ilya Loshchilov and Frank Hutter. "Decoupled Weight Decay Regularization." arXiv:1711.05101, 2017. <https://arxiv.org/abs/1711.05101>
- [16] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Re. "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness." arXiv:2205.14135, 2022. <https://arxiv.org/abs/2205.14135>
- [17] Yushi Bai et al. "IndexCache: Accelerating Sparse Attention via Cross-Layer Index Reuse." arXiv:2603.12201, 2026. <https://arxiv.org/abs/2603.12201>
- [18] Cerebras. "DeepSeek-V3.2-REAP-345B-A37B." Hugging Face model card, 2025. <https://huggingface.co/cerebras/DeepSeek-V3.2-REAP-345B-A37B>
- [19] NVIDIA. "DeepSeek-V3.2-NVFP4." Hugging Face model card, 2026. <https://huggingface.co/nvidia/DeepSeek-V3.2-NVFP4>
- [20] BlaiseAI. "blaise-sft-training-mix." Hugging Face dataset, 2026. <https://huggingface.co/datasets/BlaiseAI/blaise-sft-training-mix>
- [21] BlaiseAI. "DeepSeek-V3.2-REAP-345B-NVFP4-W4A4KV4-IndexerK8-FP8-GatedNorm-G1." Hugging Face model, 2026. <https://huggingface.co/BlaiseAI/DeepSeek-V3.2-REAP-345B-NVFP4-W4A4KV4-IndexerK8-FP8-GatedNorm-G1>